

ShiftNotes

An Agentic Operational Intelligence System for Hospitality Shift Reporting
Technical Report

Week 10 Final Deliverable
June 2026

Team: Pitchanan Lohavanichbutr, Geetanjali Kulkarni and Careen Mollel

1. Executive Summary

ShiftNotes is an agentic operational intelligence system that converts raw hospitality shift reports into concise, actionable weekly briefings for a general manager. The motivating problem is concrete: at a hospitality operation running multiple food kiosks, shift leads submit daily reports covering food quality, inventory concerns, guest feedback, and staff recognition. Before ShiftNotes, those reports accumulated unread and required a manager to review each one manually to identify patterns worth acting on.

The system addresses this by orchestrating a six-node pipeline that ingests submissions, classifies intent, detects operational signals, retrieves relevant historical context, generates a plain-English briefing, delivers it by email, and then pauses for human review. The guiding design principle is that intelligence is delivered to the manager rather than requiring the manager to go looking for it.

This report documents the system architecture, the model and tool selection rationale, the human-in-the-loop safeguards, the responsible-AI considerations, the failure and recovery behavior, and the team development process. The prototype runs end to end on a synthetic dataset of one hundred shift reports and delivers a real briefing email through the Gmail API. Each external integration is implemented to production standards with a controlled fallback, so the demonstration is reliable.

2. Problem Statement and Objectives

Shift reporting in a multi-location food service operation produces a steady stream of unstructured text. Each report contains useful operational signals, but those signals are buried in routine notes and spread across many submissions. A manager who reads every report spends a disproportionate amount of time on low-value scanning and still risks missing cross-location patterns, such as a recurring ingredient shortage affecting several kiosks in the same week.

The objective of ShiftNotes is to automate the detection and summarization of these signals while keeping a human in control of any consequential interpretation. Specifically, the system aims to:

- Ingest shift report submissions automatically from the source system.
- Detect four operational signal categories reliably across many reports.
- Aggregate and summarize those signals into a single weekly briefing.
- Deliver the briefing to the manager without requiring the manager to open a separate application.
- Pause for human review so the manager can accept, request more detail, or escalate.

The four signal categories the system detects are a chicken shortage, in which protein runs low or out during service; a poke request, in which guests ask for a menu item not currently offered; an operational issue, covering equipment failure, late delivery, or service bottlenecks; and team recognition, in which a shift lead acknowledges strong staff performance.

3. System Architecture

ShiftNotes is built on LangGraph, a framework for orchestrating stateful, multi-step agent workflows. The pipeline is modeled as a directed graph of six nodes connected by edges, with a shared state object passed from node to node. Each node reads from the state, performs its task, writes its results back, and passes the updated state forward. This design keeps each step isolated and testable while allowing the orchestration logic to control the overall flow, including the pause for human review.

3.1 The Shared State

All nodes operate on a single typed state container. It carries the raw reports, the classified intent, the detected signals, the retrieved historical context, the generated briefing, a flag indicating whether the briefing was sent, the manager's decision, an optional escalation note, and metadata including a unique run identifier, a timestamp, and an error field. The error field is central to the system's observability: any node that fails writes a description into it, and downstream nodes detect that and stop gracefully rather than crashing.

3.2 The Six Nodes

The pipeline executes the following nodes in sequence:

Node	Responsibility	Key Technology
ingest_email	Retrieve shift report submissions into state	JotForm MCP via OpenAI connector, CSV fallback
classify_intent	Route between signal detection and a future query mode	Rule-based routing on report content
detect_signals	Identify the four signal categories per report	Hybrid regex plus zero-shot classifier
retrieve_and_generate	Retrieve historical context and write the briefing	ChromaDB retrieval plus OpenAI generation
send_briefing	Deliver the briefing to the manager	Gmail API, file fallback
human_review	Pause for the manager's decision	LangGraph interrupt checkpoint

3.3 Orchestration and Control Flow

The graph entry point is the ingest node. Edges connect ingest to classify, classify to detect, detect to retrieve-and-generate, retrieve-and-generate to send, and send to human review. After human review, a conditional edge routes the flow based on the manager's decision. The graph is compiled with a checkpointer that persists state, which is the mechanism that allows the pipeline to pause at the human review node and resume later from exactly where it stopped.

3.4 Observability and Logging

Every node emits structured log lines through a shared logger. The system records a node entry and node exit for each step, a dedicated signal line each time a signal is detected, indicating whether the regex or the zero-shot classifier caught it, and an error line whenever a node fails. Each line carries the run identifier, so the complete history of a single pipeline run can be reconstructed from the logs alone. This satisfies the observability requirement and is the primary tool for debugging the system.

3.5 Walkthrough of a Single Run

The behavior of the system is most clearly understood by following one complete run. The pipeline begins at the ingest node, which loads the shift reports into state and assigns a run identifier that will tag every subsequent log line. The classify node inspects the reports and sets the intent to signal detection. The detect node iterates over every report, applies the hybrid classifier, and records each detected signal; on a representative run it finds signals in forty-five of one hundred reports and logs, for each, whether the regex or the classifier was responsible.

The retrieve-and-generate node then queries the vector store for relevant historical reports and passes the detected signals and that context to the language model, which produces the briefing. A typical briefing report the counts of each signal category, identifies the kiosks with the most flagged reports, and highlights the issues requiring attention, written in plain language a manager can act on immediately. The send node delivers this briefing to the manager's inbox through the Gmail API and sets the sent flag in state. Finally, the human review node pauses the pipeline and waits. When the manager enters a valid decision, the graph resumes, records the decision, and completes. The entire run, from ingestion through the manager's decision, is reconstructable from the logged lines that share the run identifier.

4. Data and Retrieval Strategy

The prototype operates on a synthetic dataset of one hundred shift reports spanning six kiosks across five weeks. The dataset was constructed with known signal patterns planted throughout, which allows the detection logic to be validated against a ground truth. On a representative run, the system detects signals in forty-five of the one hundred reports, a hit rate consistent with realistic operational data in which roughly half of all shifts contain something worth surfacing.

The detection logic uses a two-stage hybrid approach. The first stage applies regular expression patterns tuned for each signal category. This stage is fast, deterministic, and free, and it catches the majority of signals where the language is explicit. When the regex stage finds nothing for a given category, the second stage applies a zero-shot classification model, which can recognize a signal expressed in language the patterns do not anticipate. The logs record which stage caught each signal, providing a transparent audit trail of the detection process.

For the retrieval and generation step, historical reports are embedded and stored in ChromaDB, a vector database. When a briefing is generated, the system queries ChromaDB for reports similar to the signals detected in the current period and supplies that context to the language model. This grounds the briefing in prior operational history, allowing the summary to note whether a current issue has occurred before.

5. Model and Tool Selection Rationale

The system combines several models and tools: each selected for a specific role rather than defaulting to a single provider.

5.1 Language Model for Generation

The briefing is generated by OpenAI's gpt-4o-mini at a low temperature of 0.3. This model was chosen because the generation task is summarization and synthesis rather than open-ended creativity, where a smaller, faster, lower-cost model is appropriate. The low temperature keeps the output factual and consistent, which is essential for an operational briefing that a manager will act on. A higher temperature would risk introducing variability that does not serve the use case.

5.2 Signal Detection Models

Signal detection uses a deliberate two-model design. Regular expressions handle the common, explicit cases at zero cost and with full determinism. A zero-shot classification model serves as the fallback for cases where the language is less direct. Zero-shot classification was chosen specifically because it requires no labeled training data: the model evaluates whether a report's text matches a candidate signal label using general language understanding. This avoids the substantial effort of building and maintaining a labeled training set for a four-category classifier.

5.3 Embedding Model and Vector Store

Historical context retrieval uses the all-MiniLM-L6-v2 sentence embedding model with ChromaDB as the persistent vector store. This embedding model is compact, fast, and well suited to the short-text nature of shift reports, and ChromaDB provides a lightweight persistent store that runs locally without external infrastructure, which is appropriate for a prototype.

5.4 The Distinction Between Orchestration and Retrieval

A key architectural clarification is that LangGraph and ChromaDB serve distinct roles and are both required. LangGraph is the orchestrator: it controls which step runs, in what order, and when the pipeline pauses. ChromaDB is the memory: it stores historical reports so the retrieval step can search them. The orchestrator determines the flow of work; the vector store answers questions about operational history.

6. MCP Integration

The system satisfies the Model Context Protocol integration requirement through the ingest node, which connects to JotForm's official MCP server. JotForm is the platform through which shift leads submit their reports, so reading submissions directly from JotForm is the most logical point of integration. The earlier alternative of routing submissions through email and reading them via a Gmail integration added unnecessary steps and proved unreliable in practice.

The integration uses the OpenAI Responses API as the MCP client. The ingest node provides the JotForm MCP server as a tool to the model and instructs it in natural language to retrieve the form submissions. The model discovers and invokes the JotForm tools through the MCP protocol and returns structured data, which the node parses into the report format the rest of the pipeline expects. This is a genuine MCP integration in which the protocol is used to mediate access to an external tool, rather than a direct call to a REST endpoint.

Because the live JotForm form is not yet provisioned, the ingest node is configured to fall back to the synthetic dataset whenever the JotForm credentials are not present. This is a deliberate engineering pattern, not a workaround: the MCP path is fully implemented and production-ready and switching to live data requires only setting two environment variables. The same fallback discipline appears throughout the system and is what makes the demonstration reliable.

7. Human-in-the-Loop Design

The human review node is the system's primary human-in-the-loop checkpoint. After the briefing is delivered, the pipeline pauses using a LangGraph interrupt. The graph state persisted by the checkpoint, and execution resumes only when the manager provides a decision. The manager chooses one of three options: accept, in which the manager understands the briefing and acts on it directly; drill down, in which the manager opens a supporting interface for deeper detail; or escalate, in which the manager refers the matter to a shift lead to verify at the source, after which the pipeline can restart.

The system places the human review after delivery rather than before it. This is a deliberate choice aligned with the product principle that intelligence comes to the manager. Requiring the manager to approve a draft before it could even be read would defeat the purpose of passive delivery. The human-in-the-loop step therefore governs what the manager does in response to the briefing, not whether the briefing is permitted to reach them.

7.1 Input Validation as a Safeguard

An important safeguard was added to the human review step. In the original implementation, any input that was not one of the three valid decisions was silently treated as an acceptance. This was unsafe: a typo or an ambiguous response would be recorded as approval, and the manager would have no indication that their input had been reinterpreted. The revised implementation rejects invalid input and re-prompts the manager until a valid decision is provided, and it requires a non-empty note when the manager escalates. At the node level, an unrecognized decision is now recorded as an observable error rather than being silently coerced. This change converts a hidden failure mode into a visible, correctable one.

8. Responsible AI and Safeguards

Several aspects of the design reflect responsible-AI considerations appropriate to an operational decision-support tool.

First, the system keeps a human in control of interpretation. The pipeline detects and summarizes signals, but the manager decides what action to take, and the input validation safeguard ensures that decision is captured accurately.

Second, the system is transparent about how each signal was detected. The logs record whether a signal came from the deterministic regex stage or the probabilistic classifier, allowing a reviewer to understand the basis for any item in a briefing.

Third, the generation step is grounded in retrieved historical context and constrained to a low temperature, which reduces the risk of the model fabricating trends that are not supported by the data.

Fourth, the system handles credentials and secrets carefully. API keys and OAuth tokens are stored in environment files and credential files that are excluded from version control, and the email integration runs in a controlled testing configuration limited to known users. A known limitation, documented in the next-steps section, is that production deployment would require formal application verification or a service account rather than the testing configuration used for the prototype.

9. Failure Modes and Recovery

The system is designed to fail in observable, recoverable ways rather than crashing. Three mechanisms work together to achieve this.

The first is the shared error field. Any node that encounters an exception writes a descriptive message into the state's error field and returns, and downstream nodes check for that field and stop gracefully. This means a failure in one node does not cascade into an unhandled crash, and the failure is visible in both the state and the logs.

The second is the set of controlled fallbacks on the external integrations. When the JotForm credentials are absent, the ingest node falls back to the synthetic dataset. When the Gmail

credentials or recipient are absent, or when a send fails, the delivery node falls back to writing the briefing to a local file. In each case the pipeline continues and the failure is logged, so the demonstration never depends on an external service being reachable at a particular moment.

The third is the input validation in the human review step. An invalid decision is caught and re-prompted rather than silently accepted, which is itself a recovery path for human error.

A concrete example of a failure mode discovered and resolved during development is a data quality issue in signal detection. Empty cells in the source data were read as floating-point values rather than empty strings, which caused the text-assembly step to fail. The error was caught by the logging system, diagnosed from the logs, and resolved by coercing all field values to strings before processing. A regression test was then added to ensure the failure cannot silently reoccur. This sequence, in which the observability system surfaced the failure and a test was added to prevent recurrence, illustrates the intended failure-and-recovery discipline.

10. Testing and Continuous Integration

The system includes an automated test suite executed with pytest. The tests focus on the signal classifier, which is the most critical and most logic-dense component, and on the integrity of the shared state definition. They verify that the text-assembly step combines report fields correctly, that it handles empty values without failing, that each signal category is detected from representative text, that clean reports do not produce false positives, and that the classifier output contains all fields the downstream pipeline depends on. A dedicated regression test covers the empty-value failure described in the previous section.

A continuous integration workflow runs the test suite automatically on each push to the shared repository, providing a baseline guard against regressions. Together the tests and the workflow satisfy the continuous integration requirement and give the team confidence that changes do not silently break the core detection logic.

11. Technology Stack and Reproducible Environment

The system is implemented in Python and managed with the uv environment and dependency tool, which produces a reproducible environment from a single project definition and lock file. A new team member can clone the repository, run a single synchronization command to install the exact dependency versions, supply their own credentials, and run the pipeline. This reproducibility was important during development because it allowed every member to run the identical system rather than debugging environment differences.

The principal components of the stack are LangGraph for orchestration and the human-in-the-loop checkpoint; the OpenAI API for both briefing generation and as the client for the JotForm MCP integration; ChromaDB with a compact sentence-embedding model for historical retrieval; a zero-shot transformer model as the fallback signal classifier; and the Gmail API for briefing delivery. Configuration and secrets are supplied through an environment file that is excluded from version

control, and a corresponding example file documents the required variables without exposing any values. Credential and token files for the email integration are likewise excluded from version control and are generated locally by each user through a one-time setup script.

This separation of code from configuration means the repository contains no secrets, each team member supplies their own credentials, and the live integrations can be activated by setting environment variables without any code change.

12. Team Development Process and Lessons Learned

The project was developed by a team working in a shared repository, and the development process itself produced useful lessons that are worth documenting alongside the technical work.

The most significant lesson concerned version control discipline. For much of the project, team members committed directly to the main branch. Because multiple members edited the same files, this repeatedly produced merge conflicts, including a substantial conflict across several files that required a careful manual rebase to resolve. The conflict was resolved correctly by taking the agreed version of each code file, preserving the union of dependencies where two members had added different packages, and regenerating the dependency lock file cleanly. No work was lost, but the episode consumed time that better process would have avoided.

From this experience the team derived a workflow to adopt on future projects:

1. Each member works on a dedicated branch rather than committing directly to main.
2. Changes are merged into main through reviewed pull requests rather than direct pushes.
3. Members pull the latest main before starting work and again before pushing.
4. File ownership is divided so that members rarely edit the same file simultaneously.
5. Changes are merged in small, frequent increments rather than large batches.
6. Generated files and secrets are excluded from version control from the outset.
7. Members communicate before refactoring shared files.

Adopting the first three of these practices alone would have prevented the majority of the conflicts encountered, and they represent the single highest-value process improvement the team identified.

13. Key Architecture Decisions and Tradeoffs

Several design decisions involved genuine tradeoffs that are worth stating explicitly, as the reasoning behind them is as important as the outcome.

13.1 Hybrid Detection Versus a Single Model

The signal detector could have used the zero-shot classifier alone for every report. The hybrid design instead runs regex first and falls back to the classifier only when needed. The tradeoff is a small amount of additional code complexity in exchange for substantially lower cost and latency, since the majority of signals are caught by the free, instant regex stage, and full determinism for

the common cases. The classifier is reserved for the genuinely ambiguous reports where its language understanding adds value. This balances reliability, cost, and coverage better than either approach alone.

13.2 Review After Delivery Versus Before

The human review step was placed after the briefing is sent rather than before. The alternative, requiring approval before delivery, would give tighter control over what reaches the manager but would contradict the product principle of passive delivery and would add friction to every cycle. The chosen design accepts that the manager sees the briefing first and exercises judgment afterward, which fits the use case of a manager who should receive intelligence without having to request it.

13.3 JotForm MCP Versus Gmail Integration

Ingestion could have read submissions from email. The system instead integrates with JotForm directly through its MCP server. The tradeoff favored the shorter, more reliable path: reading from the source platform eliminates the intermediate email hop, fits the protocol-based integration requirement cleanly, and avoids the authentication difficulties encountered with the email approach. The cost is a dependency on JotForm provisioning, which is mitigated by the synthetic-data fallback.

13.4 Controlled Fallbacks Versus Hard Dependencies

Each external integration could have been written as a hard dependency that fails the run if the service is unavailable. The system instead implements controlled fallbacks on both ingestion and delivery. The tradeoff is that a demonstration may run on synthetic data or write to a file rather than always exercising the live service, but in exchange the pipeline is robust, always completes, and never fails on stage due to an external outage. For a prototype intended to be demonstrated, this reliability is the correct priority.

14. Conclusion and Next Steps

ShiftNotes demonstrates a complete agentic system that ingests real-format operational input, detects and summarizes signals, delivers a briefing by email, and pauses for human review, with each step orchestrated by LangGraph and observable through structured logging. The system satisfies the architecture, MCP integration, human-in-the-loop, observability, and continuous integration requirements, and it does so with a controlled fallback discipline that makes the demonstration reliable.

Several next steps would move the prototype toward production. The live JotForm form would be provisioned and connected, activating the MCP ingestion path on real submissions. The email integration would be moved from the testing configuration to a verified application or service account suitable for production use. The supporting drill-down interface for the manager would be completed so that the second human review option is fully functional. The signal detection

thresholds would be tuned against real submissions rather than synthetic data. Finally, the team workflow improvements described above would be adopted from the start of the next development phase.

Taken together, the prototype establishes a sound architectural foundation and a clear path to a deployable operational intelligence tool.